

Dizajn sistema **INDEKS**

Inovativni način za druženje, edukaciju i komunikaciju
studenata

Verzija: 1.0

Autori: Tijana Lazendić, Dejan Janjić, Milan Ilišković, Marko Grabas, Igor Piljagić, Mihajlo Ševa

Datum: 30.05.2024.

1. Uvod.....	3
1.1. Namjena sistema	3
1.2. Projektni ciljevi	3
1.3. Definicije, skraćenice	4
1.4. Referentni dokumenti	4
1.5. Kratak pregled dokumenta i predložene arhitekture	5
2. Arhitektura postojećeg sistema	5
3. Predložena arhitektura	5
3.1. Kratak pregled arhitekture i funkcionalnosti podsistema.....	6
3.2. Dekompozicija sistema	6
3.3. HW/SW mapiranje	8
3.4. Perzistentni sloj	10
3.5. Kontrola prava pristupa i sigurnost	11
3.6. Kontrola toka.....	13
3.7. Granična stanja sistema.....	14

1. Uvod

1.1. Namjena sistema

„INDEKS“ je sistem namijenjen studentima Elektrotehničkog fakulteta u Banjoj Luci koji omogućava efikasno dijeljenje materijala, olakšanu komunikaciju među studentima, pronalaženje odgovarajućeg instruktora, te ostale funkcionalnosti koje će olakšati iskustvo studiranja, kao što su personalizacija rasporeda i slanje obavještenja o oglasima. Korisnici putem mobilne aplikacije mogu da kreiraju nalog, koji će im omogućiti pristup ostalim funkcionalnostima sistema. Studenti moraju da imaju studentski imejl od Elektrotehničkog fakulteta kako bi uopšte mogli da kreiraju nalog. Materijali su raspoređeni po godini studija i predmetima, a dodaju ih studenti. Studenti takođe imaju mogućnost pristupa osnovnim grupama, koje su kreirane od strane administratora i razvrstane po određenom kriterijumu. Studenti imaju ponude za instrukcije u koje mogu da dodaju detaljne informacije o načinu izvođenju istih, a koje studenti mogu da recenziraju i pregledaju recenzije onih koji su pohađali. Aplikacija nudi mogućnost kreiranja personalizovanog rasporeda, a šalju se obavještenja (u vidu pop-up notifikacija) po objavljivanju oglasa na oglasnoj tabli. Administrator ima mogućnost da pregleda prijave naloga, materijala, recenzija i da ih označi kao primjerene ili neprimjerene, te da obriše, suspenduje ili aktivira suspendovani nalog, briše materijale kao i ponude za instrukcije.

1.2. Projektni ciljevi

- Niska cijena – aplikacija će biti besplatna, čime je osigurano da svi korisnici imaju mogućnost korišćenja bez finansijskih prepreka.
- Lakoća upotrebe – korisnički interfejs je jednostavan i intuitivan za korišćenje.
- Dobra dokumentacija – uz aplikaciju se prilaže detaljna specifikacija softverskih zahtjeva.
- Prilagodljivost – postoji mogućnost prilagođavanja sistema potrebama i ukusima korisnika, u vidu prilagodljivosti za oglase i raspored.
- Fleksibilnost – sistem će biti realizovana kao mobilna nativna aplikacija, tako da će pristup biti omogućen svim korisnicima mobilnih uredjaja sa internet konekcijom.
- Pouzdanost – raspoloživost i pouzdanost ovog sistema su na visokom nivou, u slučaju bilo kakve greške, ista bi bila otklonjena u što kraćem vremenskom roku.
- Sigurnost - prava pristupa određenim opcijama i podacima će biti određena na osnovu vrste naloga koju korisnik posjeduje, pri čemu neće biti moguć bilo kakav vid neovlaštenog pristupa, a sigurnost će takođe biti osigurana i u aplikativnom sloju i sloju baze podataka.
- Lakoća testiranja – kako će sistem biti realizovan u više slojeva, samim tim i proces testiranja komponenti pojedinačnih slojeva će biti olakšan.

1.3. Definicije, skraćenice

Pojam	Opis
Interfejs	Zajednička komponenta, posredstvom koje dva ili više sistema vrše razmjenu informacija, može predstavljati vezu između uređaja i osobe koja radi s njim
DBMS	Database Management System - Menadžment sistem baze podataka
UML	Unified Modeling Language - Opšti jezik za modelovanje
GUI	Graphical User Interface – Grafički korisnički interfejs, pomoću kojeg korisnik komunicira sa aplikacijom
HTTP	Hypertext Transfer Protocol - glavna i najčešća metoda prenosa informacija na webu čija je osnovna namjena omogućavanje objavljivanja i prezentacije HTML dokumenata, tj. web stranica.
Design time	Vrijeme razvoja programa
Run time	Vrijeme u kojem se izvršava program
Compile time	Vrijeme u kojem se kompajlira program
Log fajl	Datoteka dnevnika podataka koja koristi sve vrste softvera i <u>operativnih sistema</u> kako bi pratila nešto što se dogodilo, obično kompletno sa detaljima događaja, datumom i vremenom.
Servlet	Objekat programskog jezika Java koji dinamički obrađuje zahtjeve i konstruiše odgovor na zahtjev

1.4. Referentni dokumenti

Specifikacija softverskih zahtjeva – INDEKS, Inovativni način za druženje, edukaciju i komunikaciju studenata

1.5. Kratak pregled dokumenta i predložene arhitekture

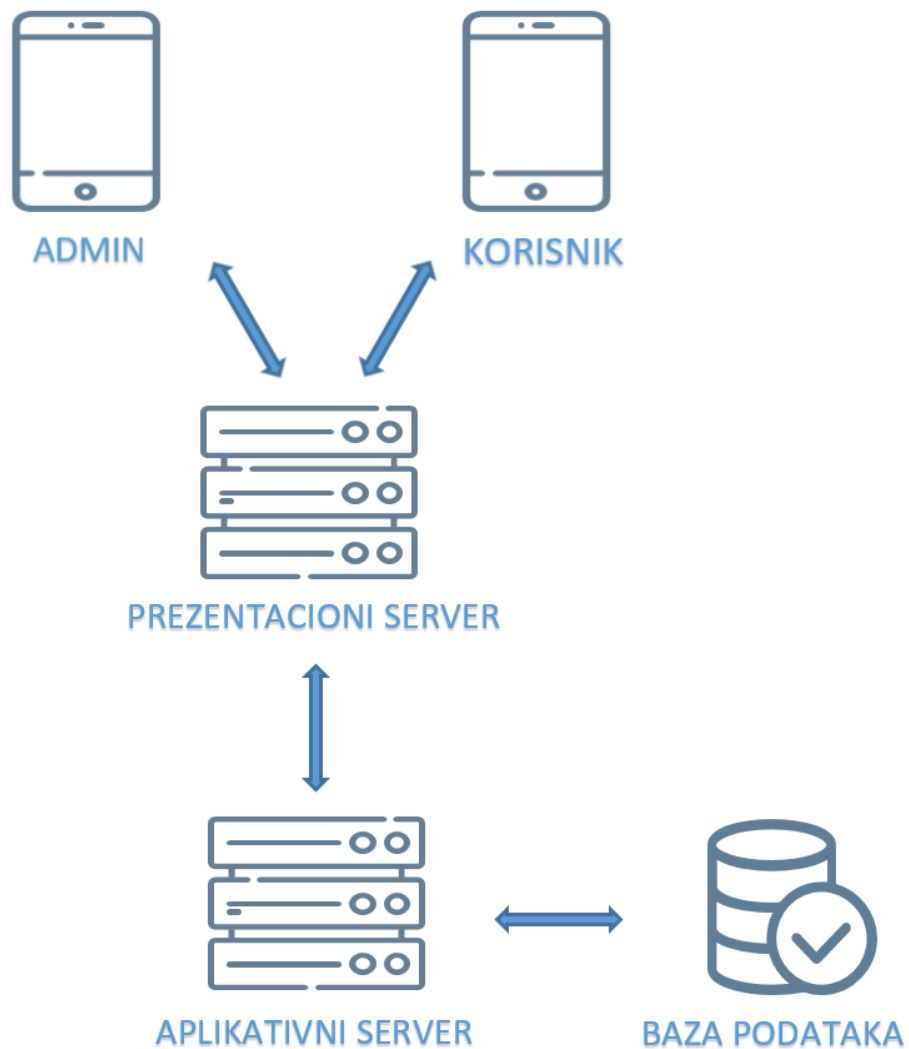
U ostatku dokumenta će biti dat pregled predložene arhitekture sistema, te dekompozicija sistema kroz dijagram paketa. Pored toga, prikazani su dijagrami komponenata i dijagrami razmještaja. Perzistentni sloj je prikazan konceptualnim modelom na visokom nivou apstrakcije. S obzirom na postojanje više tipova korisnika, ograničena su prava pristupa funkcionalnostima sistema. Ta ograničenja su prikazana u tabeli prava pristupa. Kako sistem omogućava konkurentan rad više korisnika, više riječi je dato i u toj sekciji. Što se tiče kontrole toka, takođe više riječi u datoj sekciji. Na kraju su opisani granični slučajevi upotrebe sistema.

2. Arhitektura postojećeg sistema

Trenutno ne postoji nijedan sistem koji ispunjava sve funkcionalnosti koje nudi ova aplikacija. U narednom poglavlju su navedeni detalji o predloženoj arhitekturi ovog sistema.

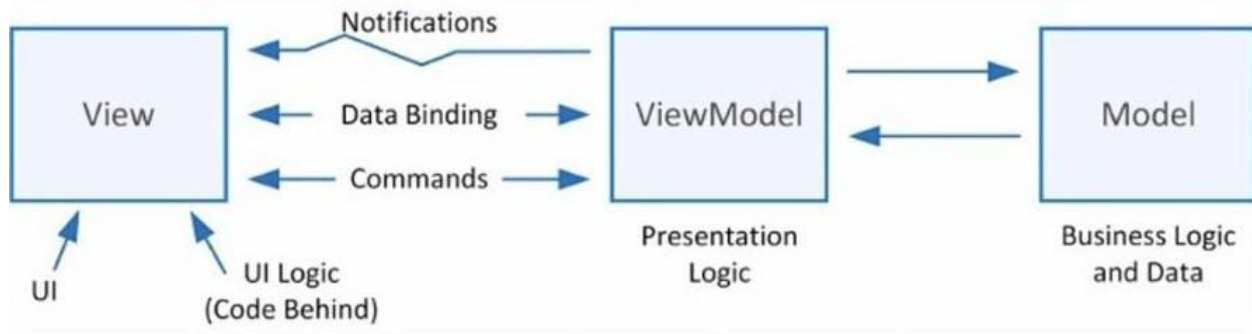
3. Predložena arhitektura

3.1. Kratak pregled arhitekture i funkcionalnosti podsistema



3.2. Dekompozicija sistema

Za realizaciju se koristi MVVM (Model, View, viewModel) arhitektura, dominantno korištena arhitektura za android aplikacije. Osnovne komponente su ilustrovane i opisane u nastavku.



Model

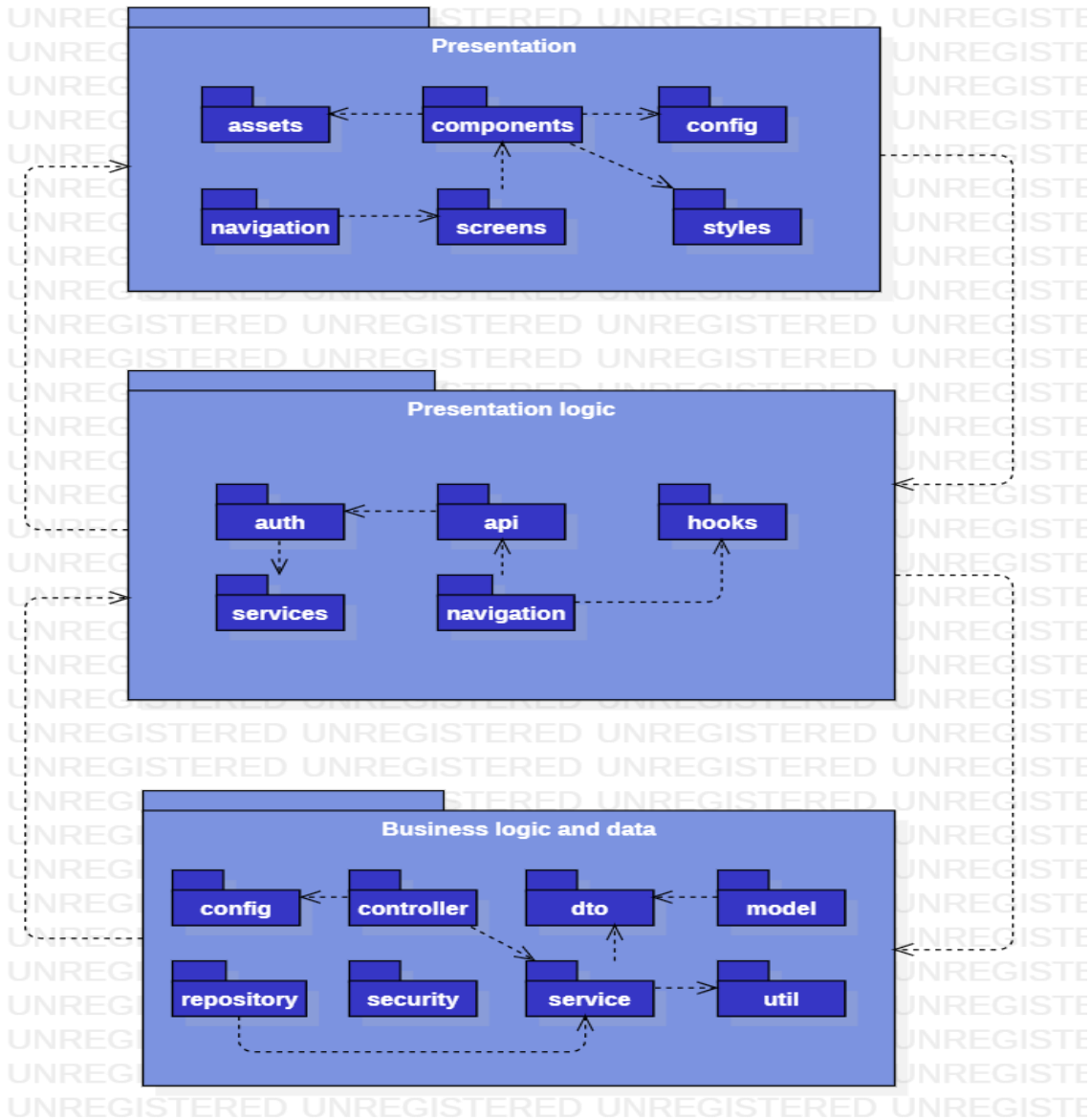
Model (model) se koristi za reprezentaciju i rad s podacima te predstavlja domenske objekte. Model je odgovoran za dohvaćanje, pohranu i obradu podataka, te za primjenu poslovne logike.

View

View (pogled) omogućava prezentaciju podataka korisniku. Pogled definiše strukturu, izgled i ponašanje korisničkog interfejsa te koristi podatke i komande iz ViewModel-a za interakciju s korisnikom.

ViewModel

ViewModel (pogledni model) služi kao posrednik između View-a i Model-a. ViewModel je odgovoran za upravljanje prezentacionom logikom i stanjem aplikacije. On dohvata podatke iz Model-a, obrađuje ih te ih izlaže View-u u obliku svojstava i komandi.



Sistem možemo, u skladu sa navedenim tipom arhitekture, dekomponovati tj. podijeliti na tri osnovna paketa: „prezentacija“, „prezentaciona logika“ i „poslovna logika i podaci“

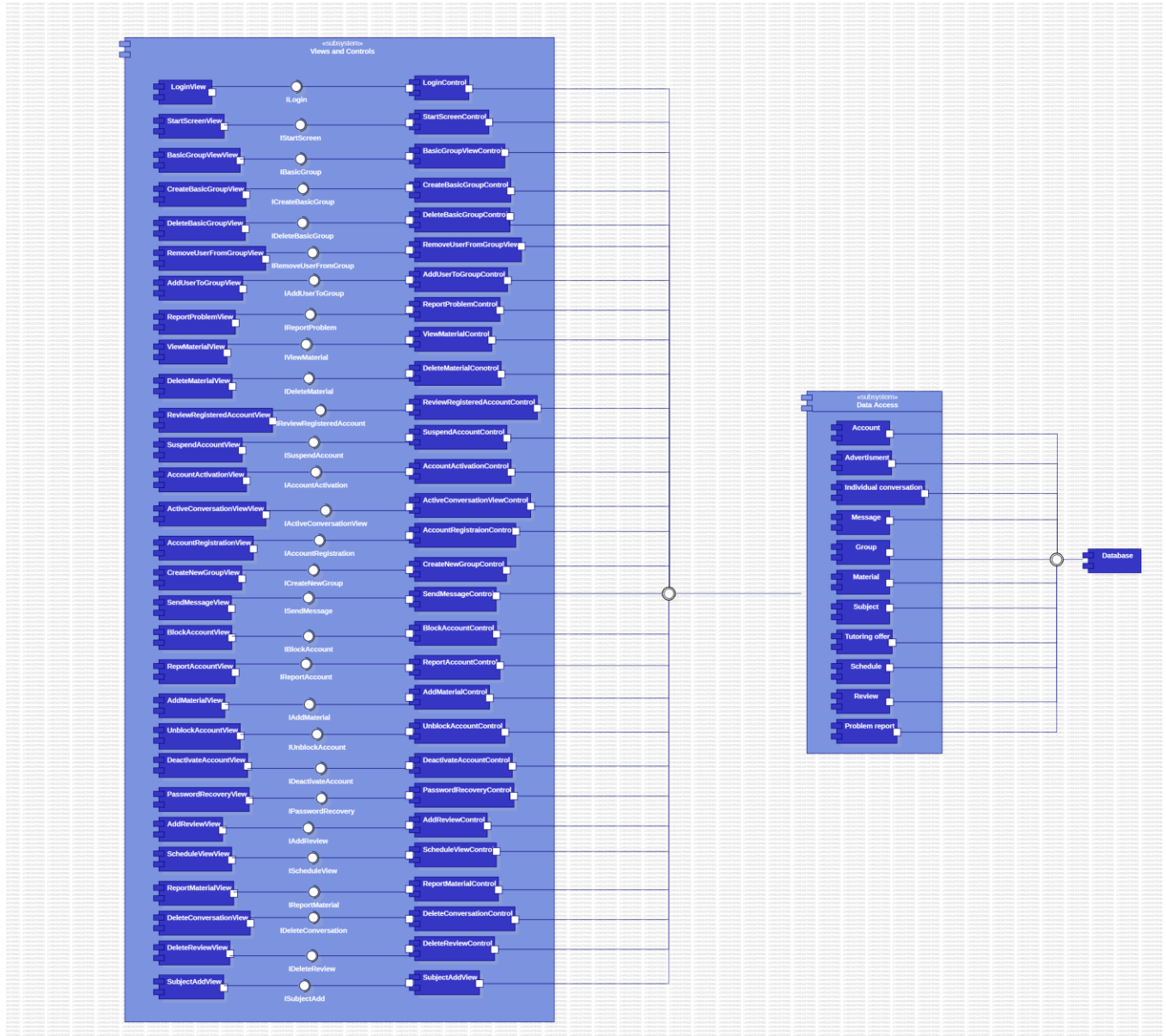
3.3. HW/SW mapiranje

UML dijagrami koji se koriste za HW/SW mapiranje su:

- o **Dijagram komponenata**
- o **Dijagram razmještaja**

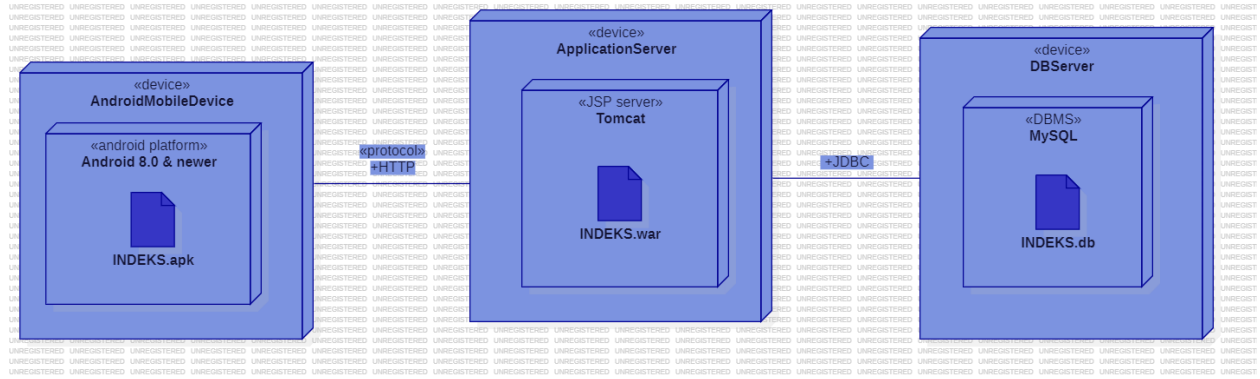
DIZAJN SISTEMA

Dijagram komponenata se koristi za modelovanje zavisnosti između komponenata (design time, compile time i run time).

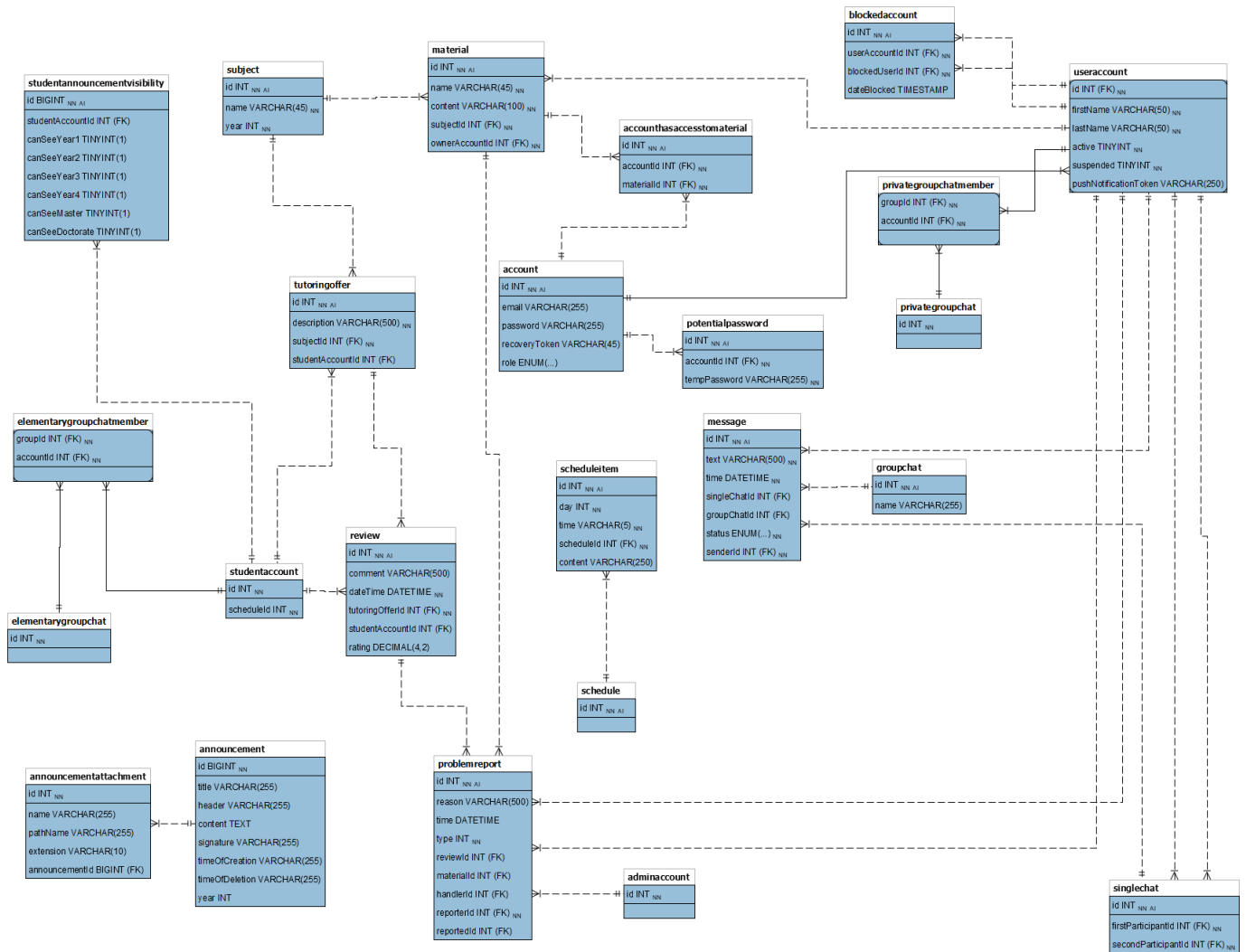


Dijagram razmještaja se koristi za modelovanje rasporeda komponenata u eksploataciji sistema (run time).

DIZAJN SISTEMA



3.4. Perzistentni sloj



3.5. Kontrola prava pristupa i sigurnost

Klasa	Operacija			
		Neregistrovani korisnik	Administrator	Student
Nalog	odjavi()		**	**
	azurirajPodatke(podaci)			**
	dohvatiNaloge()		**	
	dohvatiPodatkeONalogu(korisnickoIme)		**	**
KorisnickiNalog	dohvatiNalog(podaci)		**	
	suspenduj()		**	
	aktiviraj()		**	
	dohvatiUcesnikeURazgovoru()			**
	dohvatiProfilKorisnika()			**
	blokiraj()			**
	odblokiraj()			**
	deaktiviraj()			**
	dohvatiOpcijeZaPodesavanjeNaloga()			**

DIZAJN SISTEMA

	dohvatiListuBlokiranihNaloga()			**
	azurirajLozinku(lozinka)		**	**
PrijavaProblema	dohvatiListuSvihProblema()		**	
Recenzija	dohvatiRecenzije(ponudaInstrukcije)		**	**
PonudaZaInstrukcije	dohvatiSveMojeInstrukcije()			**
	dohvatiInstrukciju()		**	**
	kreirajPonudu()			**
	dohvatiSveInstrukcije()		**	**
Predmet	dohvatiListuPredmeta()		**	**
Materijal	dohvatiMaterijale()		**	**
	dodajMaterijale(materijali)			**
Oglas	dohvatiOglase(godina)			**
Grupa	ukloniKorisnikaIzGrupe(korisnik)			**
	dodajKorisnikaUGrupu(korisnik)			**
OsnovnaGrupa	dohvatiListuSvihOsnovnihGrupa()		**	**
	ulazakUOsnovnuGrupu()			**
PojedinačniRazgovor	dohvatiSveAktivneRazgovore()			**
	dohvatiRazgovor(korisnik1, korisnik2)			**

Raspored	dohvatiRaspored()			**
	izmjeniRaspored(podaci)			**

3.6. Kontrola toka

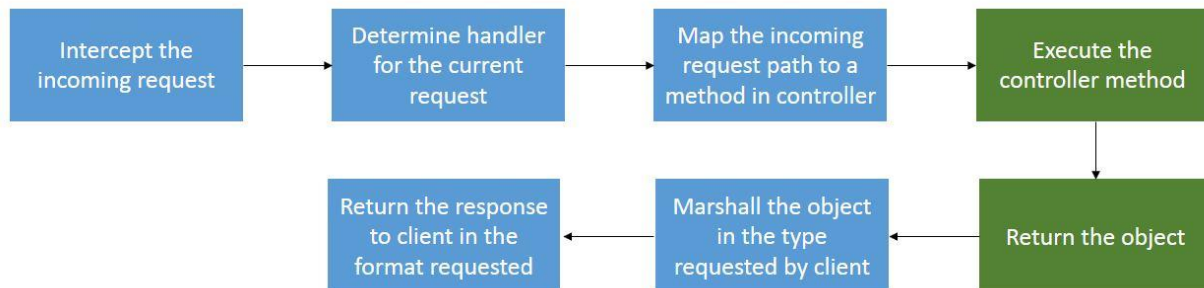
Predviđeno je da se komunikacija između sistema i korisnika obavlja asinhrono putem poruka. Konkurentni pristup serveru baze podataka će biti razriješen od strane DBMS-a upotrebom „connection pooling-a“ i dodatnih mehanizama.

Nakon što korisnik napravi zahtjev za određeni resurs, na primjer:

Zahtjev (Request): http://localhost:8080/users

Odgovor (Accept): application/json

Dolazni zahtjev obrađuje „DispatcherServlet“, koji je konfigurisan od strane Spring Boot-a (java framework). Put od zahtjeva do odgovora prikazan je na slici ispod.



Na dijagramu, zeleni blokovi predstavljaju komponente i djelove koji su implementirani od strane razvojnog tima.

Pri dohvaćanju resursa „/users“, aktivnosti se izvode u sljedećem redoslijedu:

1. DispatcherServlet presreće zahtjev za resursom na endpoint-u „http://localhost:8080/users“
2. Servlet odredi odgovarajući rukovaoc (handler) za zahtjev
3. Spring provjerava koja metoda kontrolera odgovara dolaznoj putanji „/users“
4. Izvršava se odgovarajuća metoda kontrolera (uz dodatne pozive i obradu)

5. Vraća se odgovor korisniku
6. Kako je tip poruke u jednom od standardnih formata kao što su JSON ili XML, vrši se konverzija Java objekta u tip odgovara koji je tražen od strane korisnika

3.7. Granična stanja sistema

Inicijalizacija / start sistema

Pri prvom startovanju sistema, kreira se administratorski nalog koji je odgovoran za upravljanje kompletnim sistemom. Sistem će takođe učitati osnovne podatke i konfiguracije neophodne za rad aplikacije, kao što su podaci o organizaciji materijala i osnovnih grupa. Korisnički interfejs će prikazivati ekran za logovanje i registraciju, a administrativni korisnik će imati opciju da unese inicijalne postavke sistema, kao što su dodavanje osnovnih grupa i konfiguracija komunikacionih kanala, uključujući notifikacije i chat sisteme. Procedura za startovanje uključuje inicijalizaciju baze podataka, povezivanje sa serverom, učitavanje početnih podataka i konfigurisanje komunikacionih kanala za obaveštavanje korisnika. Neregistrovani korisnici će imati ograničen pristup sistemu i biće im omogućeno da pregledaju osnovne informacije i započnu proces registracije.

Terminiranje / završetak rada

S obzirom na to da je riječ o aplikaciji kojoj se pristupa putem interneta i koja treba biti dostupna 24/7, neće dolaziti do terminiranja rada sistema osim u slučaju neočekivanih otkaza ili planiranih održavanja. Ako je potrebno da se određeni podsistemi završe, kao što su moduli za komunikaciju ili upravljanje bazom podataka, drugi podsistemi će biti obaviješteni o tome i sistem će preduzeti korake za bezbjedno završavanje rada. Promjene podataka se kontinuirano čuvaju u bazi podataka, a prije završetka rada aplikacije, svi otvoreni procesi se zatvaraju i svi podaci se sinhronizuju sa bazom. Ove procedure osiguravaju da svi korisnički podaci ostanu konzistentni i bezbjedni.

Otkazi

U slučaju otkaza ili grešaka u sistemu, sistem će detektovati pad ili grešku i pokušati automatski oporavak gdje je to moguće. U slučaju pada komunikacionih čvorova ili servera sa bazom podataka, klijent će biti obaviješten putem obavještenja u aplikaciji da trenutno nije moguće pristupiti određenim funkcionalnostima. Ako dođe do greške tokom transakcije, kao što su prenos podataka ili čuvanje informacija, sistem će osigurati da se ne dovrši nepotpuna transakcija i vratiće stanje baze na prethodno konzistentno stanje. U slučaju pada servera, sistem će uključiti proceduru obavješćavanja sistemskog administratora i inicirati procese za otklanjanje uzroka otkaza.